LAB 01C - Variables

October 26, 2026

[1]:
```
# In Python it is standard practice to import the modules
# we need at the very top of our scripts
import numpy as np
```

1 Assigning and Using Variables

Programming languages use variables (names) to refer to objects, such as numbers, that you will use over again. In Python, variables are assigned with the `=` symbol, e.g. `a = 3`.

Variables can hold almost anything, but we will usually use them to store a number (or an array of multiple numbers). Python and Numpy both have their own data types, some of which we will cover in this course. Once defined, the variable `a` can be used over and over again (see the cell below). Variables can be used to store information but you can also do calculations with variables, e.g. `c = a+b`, just like you would with numbers.

1.1 Printing Variables

We may want to check on the value stored in a variable. We can do this using the `print()` function. It is also possible to print out some text (text is referred to in programming as a "string"). You can write a string by surrounding your text with quotation marks. Run the cell below to see all this in action.

1.2 Comments

You will have noticed that the example code often contains *comments* (which start with the `#` character). You can write any text you want after the `#` character and that text will not be treated like a program to be run. It is just an informative comment for the benefit of you and anybody else reading your code. Good code contains comments that explain what the surrounding code is doing (e.g. 'calculate the length of the hypotenuse', and (for more complex code) the higher-level *purpose* of a whole larger chunk of code (e.g. 'Integrate over a circle using Simpson's rule')

[ ]:
```
# Create two new variables 'a' and 'b' and assign them integer values
a = 2
b = 3

# Print 'a' and 'b'
print(a, b)
```

```
# Update 'a' and print out 'a' and 'b' again. Note that the value printed for␣
 ↪'a' has changed
a = 4
print(a, b)

# Better: print out a string of text in order to make it clearer what the␣
 ↪numbers actually mean
print('The value of a is', a)
print('The value of b is', b)
```

Now edit the cell below, to write some code that assigns the product of `a` and `b` to a new variable `c`, and then print `a`, `b` and `c`.

[ ]:
```
# Write your Python here
```

1.3 Formatted Printing

We can mix text and numbers in a much more sophisticated way, which is helpful for generating readable output from your code. This is done by using the `.format()` method on a string of text. We will not discuss all the details this piece of relatively advanced syntax, but you should learn from examples like the ones below, and make use of them in the code you write in figure. Throughout this course (and others) you should make sure you are printing your answers to an appropriate number of significant figures, based on the significant figures of the inputs that you have been given.

[ ]:
```
a = 1.2222223
b = np.pi

# Print the variables on their own (to more significant figures than are␣
 ↪probably appropriate for your calculation)
print(a, b)

# With supporting text (still too many significant figures):
print('Variable a: {0}; Pi: {1}'.format(a,b))
# Note that {0} refers to the first variable you list after 'format' (i.e. a in␣
 ↪this case),
# {1} refers to the second variable (i.e. b in this case), etc.

# To two decimal places:
print('Variable a (2 d.p.): {0:.2f}; Pi: {1:.2f}'.format(a,b))

# To one significant figure:
print('Variable a (1 sig fig): {0:.1g}; Pi: {1:.1g}'.format(a,b))
```

1.4 Errors and Debugging

Inevitably we will make mistakes when writing our code. Try running the cell below, and you will see that it does not work. The process of fixing your own code so that it runs successfully, and

gives the correct answers, is known as *debugging*.

Can you see what these common mistakes are, and debug the code so the cell runs successfully? Edit the comments to record what the error message was and what needed to be fixed

Complete this cell after debugging and fixing the code in the next cell: This line originally read: `c = a + b / 2)`

This resulted in an error that said: ????. This was because ???, and I fixed it by ???

[ ]:
```
c = a + b / 2)
```

Complete this cell after debugging and fixing the code in the next cell: The first line originally read: `c = a + (b / 2`

This resulted in an error that said: ????. This was because ???, and I fixed it by ???. This error was a bit misleading because ???!

The second line originally read: `c = a + z`

This resulted in an error that said: ????. This was because ???, and I fixed it by ???

[ ]:
```
c = a + (b / 2

c = a + z
```

Complete this cell after debugging and fixing the code in the next cell: This line originally read `c = arccos(a)`

This resulted in an error: ????. This was because ???, and I fixed it by ???

[ ]:
```
c = arccos(a)
```

Complete this cell after debugging and fixing the code in the next cell: This line originally read `c = np.asin(a)`

This resulted in an error: ????. This was because ???, and I fixed it by ???

[ ]:
```
c = np.asin(a)
```

Complete this cell after debugging and fixing the code in the next cell: This line originally read `1stValue = a*2`

This resulted in an error: ????. This was because ???, and I fixed it by ???

[ ]:
```
1stValue = a*2
```

Not all mistakes are identified by the Python programming language as an error! Always think about the numbers your code is outputting: do they seem plausible? Can you check them by hand, for a simple example case?

Complete this cell after debugging and fixing the code in the next cell: This line originally read `hypotenuse = np.sqrt(a**2) + b**2`

This did not result in an error, but was incorrect because ???.

[ ]:
```
hypotenuse = np.sqrt(a**2) + b**2
print('A right-angled triangle with sides of length {0:.2f} and {1:.2f} has a␣
 ↪hypotenuse of length {2:.2f}'.format(a, b, hypotenuse))
```

2 Exercise 2: Using Variables (5 Marks)

A body is launched into the air at an angle $\theta = 25^\circ$ (compared to ground level) with initial velocity of $v_0 = 7.5\text{m/s}$. Assume an acceleration due to gravity of $g = 9.81\text{ms}^{-2}$.

Calculate the following:

1. Initial horizontal velocity $v_{\text{x},0}$ and vertical velocity $v_{\text{y},0}$. Do the NumPy trigonometric functions, e.g. `np.sin()` work with angles in degrees or radians? (Use an internet search to help you, if necessary). Can you find an easy way to convert between degrees and radians using NumPy?
2. Total time the body spends in the air, T. Remember that the body spends as much time going up as going down, so $v_{\text{y},0} = gT/2$.
3. Altitude of the body after $t_1 = 200\text{ms}$, $t_2 = 400\text{ms}$ and $t_3 = T$. The relevant formula is $h = v_{\text{y},0}t - \frac{1}{2}gt^2$. You then just have to change the time and re-evaluate.
4. Distance from the launch position at times $t_1 = 200\text{ms}$, $t_2 = 400\text{ms}$ and $t_3 = T$, according to $d = \sqrt{x^2 + y^2}$ where $x = v_{\text{x},0}t$ and $y = v_{\text{y},0}t - gt^2/2$.
5. What is the maximum altitude? At which distance from the launch position does the body hit the ground?

N.B. You will come back to this exercise later on, so make sure you add clear comments to your code as a reminder to yourself of what it is doing!

[ ]:
```
# Write your set-up and answer question 1 here
```

[ ]:
```
# Answer question 2 here
```

[ ]:
```
# Answer question 3 here
```

[ ]:
```
# Answer question 4 here
```

[ ]:
```
# Answer question 5 here
```